

Lista de Exercícios Práticos:

Hierarquia e Relacionamentos entre Classes em Java

Prof. Dani Juliano Czelusniak
Laboratório de Programação Orientada a Objetos

Ano Letivo de 2026

Orientações para o Laboratório:

- Todos os exercícios devem ser desenvolvidos para execução em compiladores online (como **GDB Online**, **JDoodle**) ou IDEs locais.
- Para compilar em arquivo único, declare apenas a classe `public class Main`; todas as demais classes do mesmo código-fonte não devem conter o modificador `public` (visibilidade *package-private*).
- Atente-se rigorosamente ao tipo de relacionamento exigido em cada questão (Associação Simples, Herança, Agregação ou Composição).
- Limite cada solução a aproximadamente 35 a 45 linhas de código, priorizando clareza e síntese conceitual.

Questões para Resolução

Exercício 1: Fotografia (Associação Simples)

Tema: Associação Simples (uso pontual via parâmetro)

Implemente um sistema para simular o trabalho de fotógrafos:

- a) Crie a classe **Camera** contendo o atributo privado **modelo** (String), seu respectivo construtor e o método **getModelo()**.
- b) Crie a classe **Fotografo** com o atributo privado **nome** (String) e o método **tirarFoto(Camera c)**. Esse método deve imprimir na tela que o fotógrafo está capturando uma imagem com o modelo da câmera recebida como parâmetro.
- c) No método **main**, instancie de forma independente um objeto **Camera** (ex.: “Nikon D850”) e um **Fotografo**, demonstrando o registro da foto.

Exercício 2: Projeto Sentir IA (Composição)

Tema: Relação Parte-Todo Forte (Composição e ciclo de vida dependente)

Modele um ecossistema de inteligência artificial com dependência rígida de ciclo de vida:

- a) Crie a classe **ModuloIA** com o atributo privado **tarefa** (String), construtor e o método **executar()**, que imprime a tarefa em processamento.

- b) Crie a classe `ProjetoSentirIA`. Ela deve manter um vetor interno de `ModuloIA` de tamanho 2. O construtor do projeto deve instanciar diretamente seus dois módulos (ex.: “Reconhecimento de Emoções” e “Processamento de Linguagem Natural”).
- c) Implemente no projeto o método `inicializar()`, que itera sobre os módulos e dispara suas execuções.
- d) No `main`, instancie apenas o `ProjetoSentirIA` e inicie-o, garantindo que nenhum `ModuloIA` seja instanciado externamente.

Exercício 3: Tablets Mobile (Herança)

Tema: Herança Simples (Relação “É-UM” com reaproveitamento de construtor)

Implemente a especialização de dispositivos móveis:

- a) Crie a superclasse `DispositivoMovel` com o atributo protegido `marca` (`String`), construtor e o método `ligar()`.
- b) Crie a subclasse `Tablet` herdando de `DispositivoMovel`. Adicione o atributo privado `suportaCaneta` (`boolean`) e um construtor que inicialize ambos os atributos utilizando obrigatoriamente a instrução `super(marca)`.
- c) Implemente em `Tablet` o método especializado `desenhar()`, que exibe uma mensagem indicando se o dispositivo permite ou não desenhar com caneta *Stylus*.
- d) No `main`, instancie um `Tablet`, executando tanto o método herdado `ligar()` quanto o método especializado `desenhar()`.

Exercício 4: Caminhões (Herança e Polimorfismo)

Tema: Herança com Sobrescrita de Método (*Override*) e Polimorfismo
Desenvolva a precificação de frete para veículos de transporte:

- Crie a classe base `VeiculoCarga` com o atributo protegido `toneladas` (double) e o método `calcularFrete()`, que calcula o valor base multiplicando as toneladas por R\$ 50,00.
- Crie a subclasse `Caminhao` herdando de `VeiculoCarga`. Adicione o atributo privado `eixos` (int) e sobrescreva o método `calcularFrete()`, invocando o cálculo da classe mãe (`super.calcularFrete()`) e somando uma taxa adicional de R\$ 150,00 por eixo existente.
- No `main`, instancie um `VeiculoCarga` padrão e um `Caminhao` com a mesma tonelagem (ex.: 10 toneladas), exibindo a diferença entre os fretes calculados.

Exercício 5: Bodes do Asfalto (Agregação)

Tema: Agregação (Relação Parte-Todo Fraca com ciclo de vida independente)
Modele um motoclub que reúne motociclistas associados:

- Crie a classe `Motociclista` com o atributo privado `apelido` (String), construtor e método `getApelido()`.
- Crie a classe `MotoClube` contendo o atributo `nome` (String), um vetor nativo de `Motociclista` com capacidade para 15 membros e uma variável de controle para o total de inscritos.
- Implemente o método `adicionarMembro(Motociclista m)` e o método `listarBodes()`, que imprime o nome de todos os motociclistas agregados.
- No `main`, instancie dois motociclistas externamente e agregue-os ao clube “Bodes do Asfalto”, demonstrando que os objetos existem de forma independente do clube.

Exercício 6: Playlists Musicais (Agregação)

Tema: Agregação com vetor de objetos independentes
Construa um reprodutor de áudio com lista de reprodução:

- Crie a classe `Musica` com o atributo privado `titulo` (String) e método `getTitulo()`.
- Crie a classe `Playlist` com o atributo privado `nome` (String), um vetor com capacidade para 20 músicas e o contador de músicas cadastradas.
- Desenvolva os métodos `addMusica(Musica m)` e `tocar()`, exibindo todas as faixas adicionadas em ordem de inclusão.
- No `main`, crie instâncias de músicas avulsas e adicione-as à playlist, demonstrando que a destruição ou encerramento da playlist não afetaria as instâncias das músicas.

Exercício 7: Computadores (Composição)

Tema: Composição estrita (Instanciação interna da Parte pelo Todo)
Modele a dependência estrutural entre a CPU e o computador:

- Crie a classe `Processador` com o atributo privado `modelo` (String) e o método `trabalhar()`, que relata a atividade da CPU.

- b) Crie a classe **Computador** com um atributo privado **cpu** do tipo **Processador**. No construtor de **Computador**, receba apenas o nome do processador (String) e instancie ativamente o objeto **Processador** em seu interior.
- c) Crie o método **ligar()**, que imprime a inicialização da máquina e repassa a execução para o método de trabalho do processador.
- d) No **main**, certifique-se de instanciar apenas o computador, comprovando que o cliente não tem acesso direto à criação da CPU.

Exercício 8: Reformas Prediais (Associação Simples com Referência)

Tema: Associação Unidirecional Simples de Multiplicidade 1

Modele a fiscalização técnica de uma obra predial:

- Crie a classe **Engenheiro** com o atributo privado **registroCrea** (String) e o método **getCrea()**.
- Crie a classe **Reforma** com os atributos privados **local** (String) e **responsavel** (uma referência à classe **Engenheiro**).
- Desenvolva o método **exibirPlacaDeObra()** em **Reforma**, exibindo o local da obra e o CREA do engenheiro vinculado.
- No **main**, instancie o engenheiro e vincule-o no momento da criação da reforma, mostrando a leitura cruzada de dados.

Exercício 9: Documentos e Contratos (Herança)

Tema: Herança com Especialização de Atributos e Sobrescrita

Modele o gerenciamento de contratos corporativos:

- Crie a superclasse **Documento** com o atributo protegido **titulo** (String) e o método **imprimir()**, exibindo o título do documento.
- Crie a subclasse **Contrato** herdando de **Documento**. Adicione o atributo privado **partesEnvolvidas** (String).
- Sobrescreva o método **imprimir()** para exibir um cabeçalho formal com o título herdado e a listagem das partes signatárias.
- No **main**, crie uma instância de **Documento** e uma de **Contrato**, chamando o método **imprimir()** em cada uma para comparar os resultados obtidos.

Exercício 10: Espaçonaves (Composição e Agregação Combinadas)

Tema: Integração dos Relacionamentos Estruturais em um Único Sistema

Construa uma simulação aeroespacial integrando ambos os conceitos de agregação e composição:

- Crie a classe **MotorDobra** com o método **ativar()**, indicando o acionamento dos motores de dobra espacial.
- Crie a classe **Astronauta** contendo o atributo privado **nome** (String) e o método **getNome()**.
- Crie a classe **NaveEspacial**. Ela deve conter um **MotorDobra** (instanciado em seu próprio construtor, caracterizando **Composição**) e um vetor de **Astronauta** para até 5 tripulantes com o método **embarcar(Astronauta a)** (caracterizando **Agregação**).
- Crie o método **saltar()**, que verifica se há pelo menos um tripulante a bordo para dar o comando e, em seguida, dispara o motor de dobra.
- No **main**, crie a nave, crie astronautas externamente, embarque a tripulação e execute o salto interestelar.

Bom treinamento e bom código!